

GitHub Publishing and Version Control Documentation

1. Project Overview

I developed a Linux security check script designed to automate several host-hardening and monitoring tasks in one workflow. The tool checks for system updates, enumerates listening services, reviews Docker containers when present, runs rootkit and malware checks, and analyzes authentication and system logs. This Python version evolved from an earlier Bash-based security script that documented the project's original structure and security goals. (<https://github.com/p4l0m1n0/security-check>)

2. Repository Preparation

To present the project professionally, I created a GitHub repository and organized the source code and supporting materials into a clear folder structure. The main Python script and the earlier Bash-based documentation were added to the repository, so the project's history remained visible and easy to understand. I also prepared the repository for version control by initializing Git locally and linking it to the GitHub remote.

3. Authentication and Remote Access

During setup, I encountered HTTPS authentication issues because GitHub no longer accepts account passwords for Git operations. I resolved this by generating and using SSH authentication instead, which allowed me to push code securely without relying on password-based access. I then updated the Git remote to use the SSH repository URL and verified that the connection worked correctly.

4. Git Troubleshooting and Conflict Resolution

While publishing the project, I also resolved several Git workflow issues. These included an invalid push command, a missing remote named origin, a non-fast-forward push error, and a merge conflict that occurred during rebase when both the local and remote repositories contained a README file. I resolved the conflict manually, continued the rebase, and then successfully pushed the updated branch to GitHub.

5. Final Result

The project was successfully published to GitHub and synchronized with the remote branch. A final pull confirmed that the local repository was fully up to date, which validated that the upload and rebase process had been completed correctly. The finished repository now serves as both a functional security tool and a portfolio example of scripting, documentation, Git, SSH, and troubleshooting skills.